AGENT SDK

Agent SDK overview

Build production AI agents with Claude Code as a library

Build AI agents that autonomously read files, run commands, search the web, edit code, and more. The Agent SDK gives you the same tools, agent loop, and context management that power Claude Code, programmable in Python and TypeScript.

The Agent SDK includes built-in tools for reading files, running commands, and editing code, so your agent can start working immediately without you implementing tool execution. Dive into the quickstart or explore real agents built with the SDK:

Quickstart

Build a bug-fixing agent in minutes

Example agents

Email assistant, research agent, and more

Get started

1 **Install the SDK**

TypeScript

```
npm install @anthropic-ai/claude-agent-sdk
```

Python

```
pip install claude-agent-sdk
```

The Python package requires Python 3.10 or later. If pip reports `No matching distribution found for claude-agent-sdk`, your interpreter is older than 3.10. Run `python3 --version` on macOS or Linux, or `py --version` on Windows, to check.

> The TypeScript SDK bundles a native Claude Code binary for your platform as an optional dependency, so you don't need to install Claude Code separately.

2 Set your API key

Get an API key from the **Console**, then set it as an environment variable:

```
export ANTHROPIC_API_KEY=your-api-key
```

The SDK also supports authentication via third-party API providers:

- **Amazon Bedrock**: set `CLAUDE_CODE_USE_BEDROCK=1` environment variable and configure AWS credentials
- **Claude Platform on AWS**: set `CLAUDE_CODE_USE_ANTHROPIC_AWS=1` and `ANTHROPIC_AWS_WORKSPACE_ID`, then configure AWS credentials
- **Google Vertex AI**: set `CLAUDE_CODE_USE_VERTEX=1` environment variable and configure Google Cloud credentials
- **Microsoft Azure**: set `CLAUDE_CODE_USE_FOUNDRY=1` environment variable and configure Azure credentials

See the setup guides for **Bedrock**, **Claude Platform on AWS**, **Vertex AI**, or **Azure AI Foundry** for details.

> Unless previously approved, Anthropic does not allow third party developers to offer claude.ai login or rate limits for their products, including agents built on the Claude Agent SDK. Please use the API key authentication methods described in this document instead.

3 Run your first agent

> This example creates an agent that lists files in your current directory using built-in tools.

Ready to build? Follow the **Quickstart** to create an agent that finds and fixes bugs in minutes.

Capabilities

Everything that makes Claude Code powerful is available in the SDK:

Built-in tools

```python
import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions


async def main():
    async for message in query(
        prompt="Find and fix the bug in auth.py",
        options=ClaudeAgentOptions(allowed_tools=["Read", "Edit", "Bash"]),
    ):
        print(message)  # Claude reads the file, finds the bug, edits it


asyncio.run(main())
```

Hooks

Subagents

```
npm install @anthropic-ai/claude-agent-sdk
```

MCP

```
pip install claude-agent-sdk
```

The Python package requires Python 3.10 or later. If pip reports `No matching distribution found for claude-agent-sdk`, your interpreter is older than 3.10. Run `python3 --version` on macOS or Linux, or `py --version` on Windows, to check.

Permissions

```
import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions


async def main():
    async for message in query(
        prompt="What files are in this directory?",
        options=ClaudeAgentOptions(allowed_tools=["Bash", "Glob"]),
    ):
        if hasattr(message, "result"):
            print(message.result)


asyncio.run(main())
```

Sessions

Python

Your agent can read files, run commands, and search codebases out of the box. Key tools include:

Tool	What it does
Read	Read any file in the working directory
Write	Create new files
Edit	Make precise edits to existing files
Bash	Run terminal commands, scripts, git operations
Monitor	Watch a background script and react to each output line as an event
Glob	Find files by pattern (`**/*.ts` , `src/**/*.py`)
Grep	Search file contents with regex
WebSearch	Search the web for current information
WebFetch	Fetch and parse web page content
AskUserQuestion	Ask the user clarifying questions with multiple choice options

This example creates an agent that searches your codebase for TODO comments:

Python TypeScript

TypeScript

```python
import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions


async def main():
    async for message in query(
        prompt="Find all TODO comments and create a summary",
        options=ClaudeAgentOptions(allowed_tools=["Read", "Glob", "Grep"]),
    ):
        if hasattr(message, "result"):
            print(message.result)


asyncio.run(main())
```

Python

TypeScript

Run custom code at key points in the agent lifecycle. SDK hooks use callback functions to validate, log, block, or transform agent behavior.**Available hooks:** `PreToolUse`, `PostToolUse`, `Stop`, `SessionStart`, `SessionEnd`, `UserPromptSubmit`, and more.This example logs all file changes to an audit file:

Python TypeScript

Learn more about hooks →

Python

```python
import asyncio
from datetime import datetime
from claude_agent_sdk import query, ClaudeAgentOptions, HookMatcher
```

```python
async def log_file_change(input_data, tool_use_id, context):
    file_path = input_data.get("tool_input", {}).get("file_path", "unknown")
    with open("./audit.log", "a") as f:
        f.write(f"{datetime.now()}: modified {file_path}\n")
    return {}


async def main():
    async for message in query(
        prompt="Refactor utils.py to improve readability",
        options=ClaudeAgentOptions(
            permission_mode="acceptEdits",
            hooks={
                "PostToolUse": [
                    HookMatcher(matcher="Edit|Write", hooks=[log_file_change])
                ]
            },
        ),
    ):
        if hasattr(message, "result"):
            print(message.result)


asyncio.run(main())
```

TypeScript

Python

Spawn specialized agents to handle focused subtasks. Your main agent delegates work, and subagents report back with results.Define custom agents with specialized instructions. Subagents are invoked via the Agent tool, so include `Agent` in `allowedTools` to auto-approve those invocations:

Python TypeScript

Messages from within a subagent's context include a `parent_tool_use_id` field, letting you track

which messages belong to which subagent execution.**Learn more about subagents →**

TypeScript

```
import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions, AgentDefinition


async def main():
    async for message in query(
        prompt="Use the code-reviewer agent to review this codebase",
        options=ClaudeAgentOptions(
            allowed_tools=["Read", "Glob", "Grep", "Agent"],
            agents={
                "code-reviewer": AgentDefinition(
                    description="Expert code reviewer for quality and security
reviews.",
                    prompt="Analyze code quality and suggest improvements.",
                    tools=["Read", "Glob", "Grep"],
                )
            },
        ),
    ):
        if hasattr(message, "result"):
            print(message.result)


asyncio.run(main())
```

Python

TypeScript

Connect to external systems via the Model Context Protocol: databases, browsers, APIs, and **hundreds more**.This example connects the **Playwright MCP server** to give your agent browser automation capabilities:

Python TypeScript

Learn more about MCP →

Python

```python
import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions


async def main():
    async for message in query(
        prompt="Open example.com and describe what you see",
        options=ClaudeAgentOptions(
            mcp_servers={
                "playwright": {"command": "npx", "args": ["@playwright/mcp@latest"]}
            }
        ),
    ):
        if hasattr(message, "result"):
            print(message.result)


asyncio.run(main())
```

TypeScript

Control exactly which tools your agent can use. Allow safe operations, block dangerous ones, or require approval for sensitive actions.

For interactive approval prompts and the `AskUserQuestion` tool, see Handle approvals and user input.

This example creates a read-only agent that can analyze but not modify code. `allowed_tools` pre-

approves `Read`, `Glob`, and `Grep`.

Python TypeScript

```python
import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions


async def main():
    async for message in query(
        prompt="Review this code for best practices",
        options=ClaudeAgentOptions(
            allowed_tools=["Read", "Glob", "Grep"],
        ),
    ):
        if hasattr(message, "result"):
            print(message.result)


asyncio.run(main())
```

Learn more about permissions →

Maintain context across multiple exchanges. Claude remembers files read, analysis done, and conversation history. Resume sessions later, or fork them to explore different approaches.This example captures the session ID from the first query, then resumes to continue with full context:

Python TypeScript

Learn more about sessions →

```python
import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions, SystemMessage, ResultMessage


async def main():
    session_id = None

    # First query: capture the session ID
    async for message in query(
        prompt="Read the authentication module",
        options=ClaudeAgentOptions(allowed_tools=["Read", "Glob"]),
    ):
        if isinstance(message, SystemMessage) and message.subtype == "init":
            session_id = message.data["session_id"]

    # Resume with full context from the first query
    async for message in query(
        prompt="Now find all places that call it",  # "it" = auth module
        options=ClaudeAgentOptions(resume=session_id),
    ):
        if isinstance(message, ResultMessage):
            print(message.result)


asyncio.run(main())
```

The **Anthropic Client SDK** gives you direct API access: you send prompts and implement tool execution yourself. The **Agent SDK** gives you Claude with built-in tool execution.With the Client SDK, you implement a tool loop. With the Agent SDK, Claude handles it:

Python TypeScript

```python
# Client SDK: You implement the tool loop
response = client.messages.create(...)
while response.stop_reason == "tool_use":
    result = your_tool_executor(response.tool_use)
    response = client.messages.create(tool_result=result, **params)

# Agent SDK: Claude handles tools autonomously
async for message in query(prompt="Fix the bug in auth.py"):
    print(message)
```

Same capabilities, different interface:

Use case	Best choice
Interactive development	CLI
CI/CD pipelines	SDK
Custom applications	SDK
One-off tasks	CLI
Production automation	SDK

Many teams use both: CLI for daily development, SDK for production. Workflows translate directly between them.

Managed Agents is a hosted REST API: Anthropic runs the agent and the sandbox, and your application sends events and streams back results. The **Agent SDK** is a library that runs the agent loop inside your own process.

	Agent SDK	Managed Agents
Runs in	Your process, your infrastructure	Anthropic-managed infrastructure
Interface	Python or TypeScript library	REST API
Agent works on	Files on your infrastructure	A managed sandbox per session
Session state	JSONL on your filesystem	Anthropic-hosted event log
Custom tools	In-process Python or TypeScript functions	Claude triggers the tool; you execute and return results
Best for	Local prototyping, agents that work directly on your filesystem and services	Production agents without operating sandbox or session infrastructure, long-running and asynchronous sessions

A common path is to prototype with the Agent SDK locally, then move to Managed Agents for production.

Claude Code features

The SDK also supports Claude Code's filesystem-based configuration. With default options the SDK loads these from `.claude/` in your working directory and `~/.claude/`. To restrict which sources load, set `setting_sources` (Python) or `settingSources` (TypeScript) in your options.

Feature	Description	Location
Skills	Specialized capabilities Claude uses automatically or you invoke with `/name`	`.claude/skills/*/SKILL.md`
Commands	Custom commands in the legacy format. Use skills for new custom commands	`.claude/commands/*.md`
Memory	Project context and instructions	`CLAUDE.md` or `.claude/CLAUDE.md`
Plugins	Extend with skills, agents, hooks, and MCP servers	Programmatic via `plugins` option

Compare the Agent SDK to other Claude tools

The Claude Platform offers multiple ways to build with Claude. Here's how the Agent SDK fits in:

Agent SDK vs Client SDK

```python
import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions


async def main():
    async for message in query(
        prompt="Find and fix the bug in auth.py",
        options=ClaudeAgentOptions(allowed_tools=["Read", "Edit", "Bash"]),
    ):
        print(message)  # Claude reads the file, finds the bug, edits it


asyncio.run(main())
```

Agent SDK vs Claude Code CLI

Agent SDK vs Managed Agents

```
npm install @anthropic-ai/claude-agent-sdk
```

Python

```
pip install claude-agent-sdk
```

The Python package requires Python 3.10 or later. If pip reports `No matching distribution found for claude-agent-sdk`, your interpreter is older than 3.10. Run `python3 --version` on macOS or Linux, or `py --version` on Windows, to check.

TypeScript

```python
import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions

async def main():
    async for message in query(
        prompt="What files are in this directory?",
        options=ClaudeAgentOptions(allowed_tools=["Bash", "Glob"]),
    ):
        if hasattr(message, "result"):
            print(message.result)

asyncio.run(main())
```

```python
import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions

async def main():
    async for message in query(
        prompt="Find all TODO comments and create a summary",
        options=ClaudeAgentOptions(allowed_tools=["Read", "Glob", "Grep"]),
    ):
        if hasattr(message, "result"):
            print(message.result)

asyncio.run(main())
```

```python
import asyncio
from datetime import datetime
from claude_agent_sdk import query, ClaudeAgentOptions, HookMatcher


async def log_file_change(input_data, tool_use_id, context):
    file_path = input_data.get("tool_input", {}).get("file_path", "unknown")
    with open("./audit.log", "a") as f:
        f.write(f"{datetime.now()}: modified {file_path}\n")
    return {}


async def main():
    async for message in query(
        prompt="Refactor utils.py to improve readability",
        options=ClaudeAgentOptions(
            permission_mode="acceptEdits",
            hooks={
                "PostToolUse": [
                    HookMatcher(matcher="Edit|Write", hooks=[log_file_change])
                ]
            },
        ),
    ):
        if hasattr(message, "result"):
            print(message.result)


asyncio.run(main())
```

```python
import asyncio
```

```python
from claude_agent_sdk import query, ClaudeAgentOptions, AgentDefinition


async def main():
    async for message in query(
        prompt="Use the code-reviewer agent to review this codebase",
        options=ClaudeAgentOptions(
            allowed_tools=["Read", "Glob", "Grep", "Agent"],
            agents={
                "code-reviewer": AgentDefinition(
                    description="Expert code reviewer for quality and security
reviews.",
                    prompt="Analyze code quality and suggest improvements.",
                    tools=["Read", "Glob", "Grep"],
                )
            },
        ),
    ):
        if hasattr(message, "result"):
            print(message.result)


asyncio.run(main())
```

```python
import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions


async def main():
    async for message in query(
        prompt="Open example.com and describe what you see",
        options=ClaudeAgentOptions(
            mcp_servers={
                "playwright": {"command": "npx", "args": ["@playwright/mcp@latest"]}
            }
        ),
    ):
        if hasattr(message, "result"):
            print(message.result)
```

```
asyncio.run(main())
```

```
import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions

async def main():
    async for message in query(
        prompt="Review this code for best practices",
        options=ClaudeAgentOptions(
            allowed_tools=["Read", "Glob", "Grep"],
        ),
    ):
        if hasattr(message, "result"):
            print(message.result)

asyncio.run(main())
```

```
import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions, SystemMessage, ResultMessage

async def main():
    session_id = None
```

```python
    # First query: capture the session ID
    async for message in query(
        prompt="Read the authentication module",
        options=ClaudeAgentOptions(allowed_tools=["Read", "Glob"]),
    ):
        if isinstance(message, SystemMessage) and message.subtype == "init":
            session_id = message.data["session_id"]

    # Resume with full context from the first query
    async for message in query(
        prompt="Now find all places that call it",  # "it" = auth module
        options=ClaudeAgentOptions(resume=session_id),
    ):
        if isinstance(message, ResultMessage):
            print(message.result)


asyncio.run(main())
```

```python
# Client SDK: You implement the tool loop
response = client.messages.create(...)
while response.stop_reason == "tool_use":
    result = your_tool_executor(response.tool_use)
    response = client.messages.create(tool_result=result, **params)

# Agent SDK: Claude handles tools autonomously
async for message in query(prompt="Fix the bug in auth.py"):
    print(message)
```

```python
import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions


async def main():
    async for message in query(
        prompt="Find all TODO comments and create a summary",
        options=ClaudeAgentOptions(allowed_tools=["Read", "Glob", "Grep"]),
    ):
        if hasattr(message, "result"):
            print(message.result)


asyncio.run(main())
```

```python
import asyncio
from datetime import datetime
from claude_agent_sdk import query, ClaudeAgentOptions, HookMatcher


async def log_file_change(input_data, tool_use_id, context):
    file_path = input_data.get("tool_input", {}).get("file_path", "unknown")
    with open("./audit.log", "a") as f:
        f.write(f"{datetime.now()}: modified {file_path}\n")
    return {}


async def main():
    async for message in query(
        prompt="Refactor utils.py to improve readability",
        options=ClaudeAgentOptions(
            permission_mode="acceptEdits",
            hooks={
                "PostToolUse": [
                    HookMatcher(matcher="Edit|Write", hooks=[log_file_change])
                ]
            },
```

```
        ),
    ):
        if hasattr(message, "result"):
            print(message.result)


asyncio.run(main())
```

```
import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions, AgentDefinition


async def main():
    async for message in query(
        prompt="Use the code-reviewer agent to review this codebase",
        options=ClaudeAgentOptions(
            allowed_tools=["Read", "Glob", "Grep", "Agent"],
            agents={
                "code-reviewer": AgentDefinition(
                    description="Expert code reviewer for quality and security
reviews.",
                    prompt="Analyze code quality and suggest improvements.",
                    tools=["Read", "Glob", "Grep"],
                )
            },
        ),
    ):
        if hasattr(message, "result"):
            print(message.result)


asyncio.run(main())
```

```python
import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions


async def main():
    async for message in query(
        prompt="Open example.com and describe what you see",
        options=ClaudeAgentOptions(
            mcp_servers={
                "playwright": {"command": "npx", "args": ["@playwright/mcp@latest"]}
            }
        ),
    ):
        if hasattr(message, "result"):
            print(message.result)


asyncio.run(main())
```

```python
import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions


async def main():
    async for message in query(
        prompt="Review this code for best practices",
        options=ClaudeAgentOptions(
            allowed_tools=["Read", "Glob", "Grep"],
        ),
    ):
        if hasattr(message, "result"):
            print(message.result)
```

```
asyncio.run(main())
```

```
import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions, SystemMessage, ResultMessage


async def main():
    session_id = None

    # First query: capture the session ID
    async for message in query(
        prompt="Read the authentication module",
        options=ClaudeAgentOptions(allowed_tools=["Read", "Glob"]),
    ):
        if isinstance(message, SystemMessage) and message.subtype == "init":
            session_id = message.data["session_id"]

    # Resume with full context from the first query
    async for message in query(
        prompt="Now find all places that call it",  # "it" = auth module
        options=ClaudeAgentOptions(resume=session_id),
    ):
        if isinstance(message, ResultMessage):
            print(message.result)


asyncio.run(main())
```

Your agent can read files, run commands, and search codebases out of the box. Key tools include:

Tool	What it does
Read	Read any file in the working directory
Write	Create new files
Edit	Make precise edits to existing files
Bash	Run terminal commands, scripts, git operations
Monitor	Watch a background script and react to each output line as an event
Glob	Find files by pattern (`**/*.ts` , `src/**/*.py`)
Grep	Search file contents with regex
WebSearch	Search the web for current information
WebFetch	Fetch and parse web page content
AskUserQuestion	Ask the user clarifying questions with multiple choice options

This example creates an agent that searches your codebase for TODO comments:

```python
import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions


async def main():
    async for message in query(
        prompt="Find all TODO comments and create a summary",
        options=ClaudeAgentOptions(allowed_tools=["Read", "Glob", "Grep"]),
    ):
        if hasattr(message, "result"):
            print(message.result)


asyncio.run(main())
```

Run custom code at key points in the agent lifecycle. SDK hooks use callback functions to validate, log, block, or transform agent behavior.**Available hooks:** `PreToolUse`, `PostToolUse`, `Stop`, `SessionStart`, `SessionEnd`, `UserPromptSubmit`, and more.This example logs all file changes to an audit file:

Learn more about hooks →

```python
import asyncio
from datetime import datetime
from claude_agent_sdk import query, ClaudeAgentOptions, HookMatcher


async def log_file_change(input_data, tool_use_id, context):
    file_path = input_data.get("tool_input", {}).get("file_path", "unknown")
    with open("./audit.log", "a") as f:
        f.write(f"{datetime.now()}: modified {file_path}\n")
    return {}


async def main():
    async for message in query(
        prompt="Refactor utils.py to improve readability",
        options=ClaudeAgentOptions(
            permission_mode="acceptEdits",
            hooks={
                "PostToolUse": [
                    HookMatcher(matcher="Edit|Write", hooks=[log_file_change])
                ]
            },
        ),
    ):
        if hasattr(message, "result"):
            print(message.result)


asyncio.run(main())
```

Spawn specialized agents to handle focused subtasks. Your main agent delegates work, and subagents report back with results.Define custom agents with specialized instructions. Subagents are invoked via the Agent tool, so include `Agent` in `allowedTools` to auto-approve those invocations:

Messages from within a subagent's context include a `parent_tool_use_id` field, letting you track

which messages belong to which subagent execution.**Learn more about subagents →**

```
import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions, AgentDefinition


async def main():
    async for message in query(
        prompt="Use the code-reviewer agent to review this codebase",
        options=ClaudeAgentOptions(
            allowed_tools=["Read", "Glob", "Grep", "Agent"],
            agents={
                "code-reviewer": AgentDefinition(
                    description="Expert code reviewer for quality and security
reviews.",
                    prompt="Analyze code quality and suggest improvements.",
                    tools=["Read", "Glob", "Grep"],
                )
            },
        ),
    ):
        if hasattr(message, "result"):
            print(message.result)


asyncio.run(main())
```

Connect to external systems via the Model Context Protocol: databases, browsers, APIs, and **hundreds more**.This example connects the **Playwright MCP server** to give your agent browser automation capabilities:

Learn more about MCP →

```python
import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions


async def main():
    async for message in query(
        prompt="Open example.com and describe what you see",
        options=ClaudeAgentOptions(
            mcp_servers={
                "playwright": {"command": "npx", "args": ["@playwright/mcp@latest"]}
            }
        ),
    ):
        if hasattr(message, "result"):
            print(message.result)


asyncio.run(main())
```

Control exactly which tools your agent can use. Allow safe operations, block dangerous ones, or require approval for sensitive actions.

> For interactive approval prompts and the `AskUserQuestion` tool, see **Handle approvals and user input**.

This example creates a read-only agent that can analyze but not modify code. `allowed_tools` pre-approves `Read`, `Glob`, and `Grep`.

Learn more about permissions →

```python
import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions


async def main():
    async for message in query(
        prompt="Review this code for best practices",
        options=ClaudeAgentOptions(
            allowed_tools=["Read", "Glob", "Grep"],
        ),
    ):
        if hasattr(message, "result"):
            print(message.result)


asyncio.run(main())
```

Maintain context across multiple exchanges. Claude remembers files read, analysis done, and conversation history. Resume sessions later, or fork them to explore different approaches.This example captures the session ID from the first query, then resumes to continue with full context:

Learn more about sessions →

```python
import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions, SystemMessage, ResultMessage


async def main():
    session_id = None
```

```python
    # First query: capture the session ID
    async for message in query(
        prompt="Read the authentication module",
        options=ClaudeAgentOptions(allowed_tools=["Read", "Glob"]),
    ):
        if isinstance(message, SystemMessage) and message.subtype == "init":
            session_id = message.data["session_id"]

    # Resume with full context from the first query
    async for message in query(
        prompt="Now find all places that call it",  # "it" = auth module
        options=ClaudeAgentOptions(resume=session_id),
    ):
        if isinstance(message, ResultMessage):
            print(message.result)


asyncio.run(main())
```

The **Anthropic Client SDK** gives you direct API access: you send prompts and implement tool execution yourself. The **Agent SDK** gives you Claude with built-in tool execution.With the Client SDK, you implement a tool loop. With the Agent SDK, Claude handles it:

Python TypeScript

```python
# Client SDK: You implement the tool loop
response = client.messages.create(...)
while response.stop_reason == "tool_use":
    result = your_tool_executor(response.tool_use)
    response = client.messages.create(tool_result=result, **params)

# Agent SDK: Claude handles tools autonomously
async for message in query(prompt="Fix the bug in auth.py"):
```

```
        print(message)
```

Same capabilities, different interface:

Use case	Best choice
Interactive development	CLI
CI/CD pipelines	SDK
Custom applications	SDK
One-off tasks	CLI
Production automation	SDK

Many teams use both: CLI for daily development, SDK for production. Workflows translate directly between them.

Managed Agents is a hosted REST API: Anthropic runs the agent and the sandbox, and your application sends events and streams back results. The **Agent SDK** is a library that runs the agent loop inside your own process.

	Agent SDK	Managed Agents
Runs in	Your process, your infrastructure	Anthropic-managed infrastructure
Interface	Python or TypeScript library	REST API
Agent works on	Files on your infrastructure	A managed sandbox per session
Session state	JSONL on your filesystem	Anthropic-hosted event log
Custom tools	In-process Python or TypeScript functions	Claude triggers the tool; you execute and return results
Best for	Local prototyping, agents that work	Production agents without operating sandbox or

	Agent SDK	Managed Agents
	directly on your filesystem and services	session infrastructure, long-running and asynchronous sessions

A common path is to prototype with the Agent SDK locally, then move to Managed Agents for production.

Changelog

View the full changelog for SDK updates, bug fixes, and new features:

- **TypeScript SDK**: **view CHANGELOG.md**
- **Python SDK**: **view CHANGELOG.md**

Reporting bugs

If you encounter bugs or issues with the Agent SDK:

- **TypeScript SDK**: **report issues on GitHub**
- **Python SDK**: **report issues on GitHub**

Branding guidelines

For partners integrating the Claude Agent SDK, use of Claude branding is optional. When referencing Claude in your product:

Allowed:

- “Claude Agent” (preferred for dropdown menus)
- “Claude” (when within a menu already labeled “Agents”)
- ” Powered by Claude” (if you have an existing agent name)

Not permitted:

- “Claude Code” or “Claude Code Agent”
- Claude Code-branded ASCII art or visual elements that mimic Claude Code

Your product should maintain its own branding and not appear to be Claude Code or any Anthropic product. For questions about branding compliance, contact the Anthropic **sales team**.

License and terms

Use of the Claude Agent SDK is governed by **Anthropic's Commercial Terms of Service**, including when you use it to power products and services that you make available to your own customers and end users, except to the extent a specific component or dependency is covered by a different license as indicated in that component's LICENSE file.

Next steps

Quickstart

Build an agent that finds and fixes bugs in minutes

Example agents

Email assistant, research agent, and more

TypeScript SDK

Full TypeScript API reference and examples

Python SDK

Full Python API reference and examples